

Object Detection using HOG Features and Sliding Window

Object Detection using HOG Features and Sliding Window

Name: Yasser ali mohamed mohamed

ID: 2305268

Introduction

Object detection is one of the fundamental tasks in computer vision, where the goal is to automatically locate and identify objects inside images. This project implements a classical object detection pipeline using **Histogram of Oriented Gradients (HOG)** features combined with a **Support Vector Machine (SVM)** classifier.

The system processes images through several stages including **data preprocessing, feature extraction, dataset splitting, model training, and object detection using sliding windows and image pyramids.**

This structured pipeline helps ensure that the data is consistent and suitable for machine learning models, which improves the reliability of the detection results.

Dataset Analysis

Before training the model, it is important to understand the structure of the dataset and the type of data being used. In this project, the dataset consists of two main categories of images: **images that contain the target object** and **images that do not contain the object.**

The images that contain the object represent the **positive class**, while the images without the object represent the **negative class**. This type of dataset is commonly used in object detection problems, where the model needs to learn how to distinguish between the presence and absence of a specific object.

By organizing the dataset into these two categories, the system can learn the visual patterns that characterize the object and differentiate them from other background patterns. This structure is essential for training the classifier to correctly recognize the object during the

detection stage.



Image Orientation Correction

Some images in the dataset may have extremely large widths compared to their heights, which indicates that the image orientation may not be suitable for the processing pipeline.

To solve this issue, images whose width exceeds a specific threshold are **transposed**, effectively swapping width and height. The corrected images are then saved into a new folder.

This preprocessing step ensures that images have more consistent orientations.

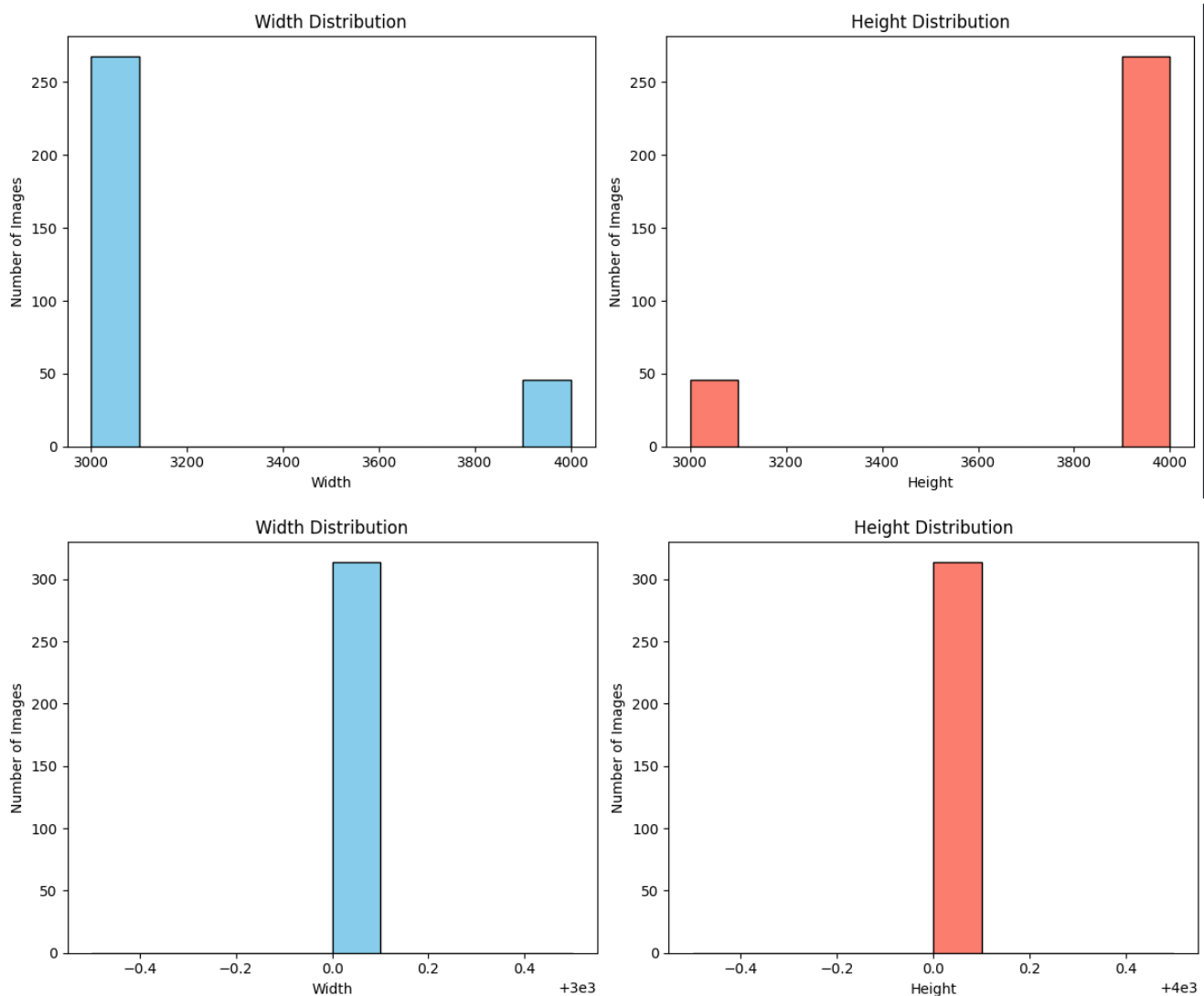


Image Resizing

Since machine learning algorithms require consistent input dimensions, all images are resized to a fixed size of **128 × 128 pixels**.

This step standardizes the dataset so that every image has the same resolution before feature extraction.

Maintaining a fixed size is essential because HOG feature vectors depend on image dimensions.

```
import os
import cv2

def resize_all_images(original_folder, output_folder, size=(128,128)):
    os.makedirs(output_folder, exist_ok=True)

    for file in os.listdir(original_folder):
        if file.lower().endswith((".jpg", ".jpeg", ".png", ".bmp")):
            img_path = os.path.join(original_folder, file)
            img = cv2.imread(img_path)
            if img is not None:
                resized_img = cv2.resize(img, size)
                cv2.imwrite(os.path.join(output_folder, file), resized_img)
    print("Resizing complete.")

original_folder = "../data/same_shape_1"
output_folder = "../data/resized_img_2"

resize_all_images(original_folder, output_folder, size=(128,128))
```

Python

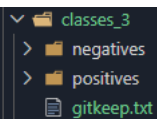
Dataset Organization into Classes

After preprocessing, the dataset is divided into two classes:

- **Positive images** containing the target object
- **Negative images** containing no object

The separation is done automatically based on the filename pattern.

Organizing images into class folders simplifies the training process because machine learning models require labeled data.



Dataset Splitting

The dataset is divided into three subsets:

- **Training set**
- **Validation set**
- **Test set**

The split is performed using stratified sampling to maintain balanced class distribution across the subsets.

The training set is used to train the model, the validation set is used to evaluate and tune the model, and the test set is used to measure final performance.

This separation is important to prevent overfitting.



Feature Extraction using HOG

After organizing the dataset, features are extracted using the **Histogram of Oriented Gradients (HOG)** method.

HOG works by analyzing the **direction of gradients in small regions of the image**. These gradients capture edge and shape information which are important for recognizing objects.

Each image is converted to grayscale and resized before computing the HOG descriptor.

The extracted HOG vectors represent the image numerically and are used as input for the machine learning classifier.

```
Train features shape: (219, 8100)
Valid features shape: (47, 8100)
Test features shape: (48, 8100)
```

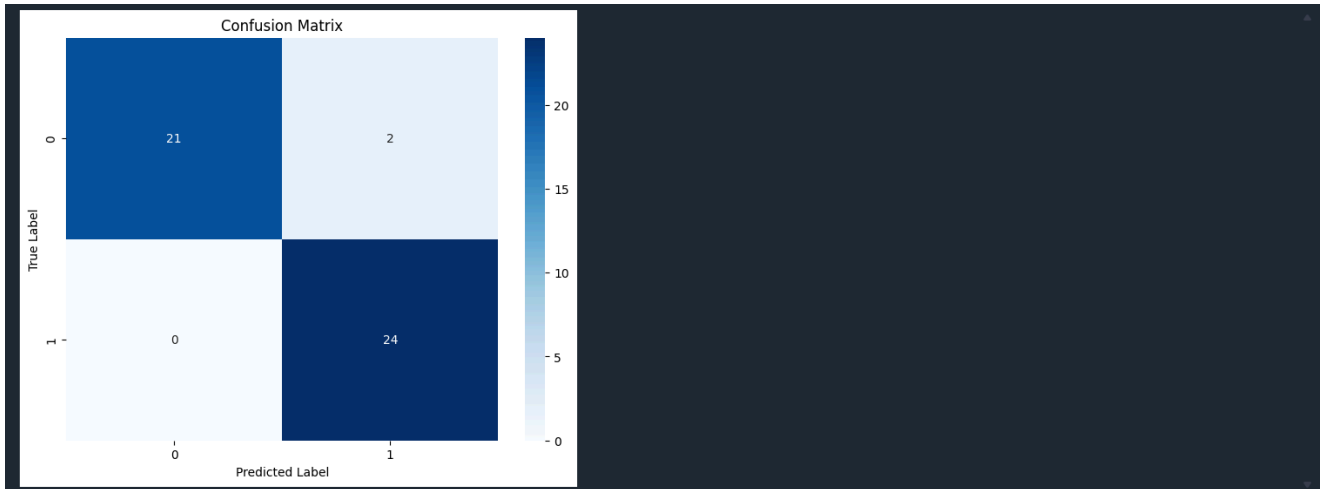
Model Evaluation

To evaluate the model performance, a **confusion matrix** is generated. This matrix compares the predicted labels with the true labels from the validation dataset.

The confusion matrix provides insight into:

- True positives
- True negatives
- False positives
- False negatives

This evaluation helps understand where the model makes mistakes.



Object Detection using Sliding Window

After training the classifier, the system applies object detection on larger images.

A **sliding window technique** is used, where a fixed-size window scans the image step by step. For each window:

1. HOG features are extracted
2. The trained SVM classifier predicts whether the object exists inside that window

If the classifier confidence exceeds a threshold, the window is considered a detection.

This method allows the system to search for objects across the entire image.

```
import os
import cv2
import numpy as np
from skimage.feature import hog

def extract_hog_features(image, resize_dim=(128,128)):
    if len(image.shape) > 2:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    image = cv2.resize(image, resize_dim)
    features = hog(image,
                    orientations=9,
                    pixels_per_cell=(8,8),
                    cells_per_block=(2,2),
                    block_norm='L2-Hys',
                    visualize=False)
    return features
```

Image Pyramid for Multi-Scale Detection

Objects may appear at different sizes within an image. To detect objects at multiple scales, the system uses an **image pyramid**.

The image is repeatedly resized into smaller versions, and the sliding window is applied at each scale.

This technique enables the model to detect both large and small objects.

```
def pyramid(image, scale=1.5, min_size=(128,128)):
    yield image
    while True:
        w = int(image.shape[1] / scale)
        h = int(image.shape[0] / scale)
        if w < min_size[0] or h < min_size[1]:
            break
        image = cv2.resize(image, (w,h))
        yield image
```

Python

Non-Maximum Suppression (NMS)

During detection, multiple overlapping bounding boxes may be generated around the same object.

To remove redundant detections, the project applies **Non-Maximum Suppression (NMS)**. This algorithm keeps the bounding box with the highest confidence score and removes overlapping boxes with lower scores.

This step improves the clarity of the final detection results.

```
def non_max_suppression(detections, overlap_thresh=0.3):
    if len(detections) == 0:
        return []
    boxes = np.array([(x, y, x+w, y+h) for (x, y, w, h, _) in detections])
    scores = np.array([s for (_, _, _, _, s) in detections])
    x1, y1, x2, y2 = boxes[:,0], boxes[:,1], boxes[:,2], boxes[:,3]
    areas = (x2 - x1 + 1) * (y2 - y1 + 1)
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]
        keep.append(i)
        xx1 = np.maximum(x1[i], x1[order[1:]])
        yy1 = np.maximum(y1[i], y1[order[1:]])
        xx2 = np.minimum(x2[i], x2[order[1:]])
        yy2 = np.minimum(y2[i], y2[order[1:]])
        inter = np.maximum(0, xx2 - xx1 + 1) * np.maximum(0, yy2 - yy1 + 1)
        iou = inter / (areas[i] + areas[order[1:]] - inter)
        order = order[np.where(iou <= overlap_thresh)[0] + 1]
    return [(detections[i][0], detections[i][1], detections[i][2], detections[i][3]) for i in keep]
```

Python

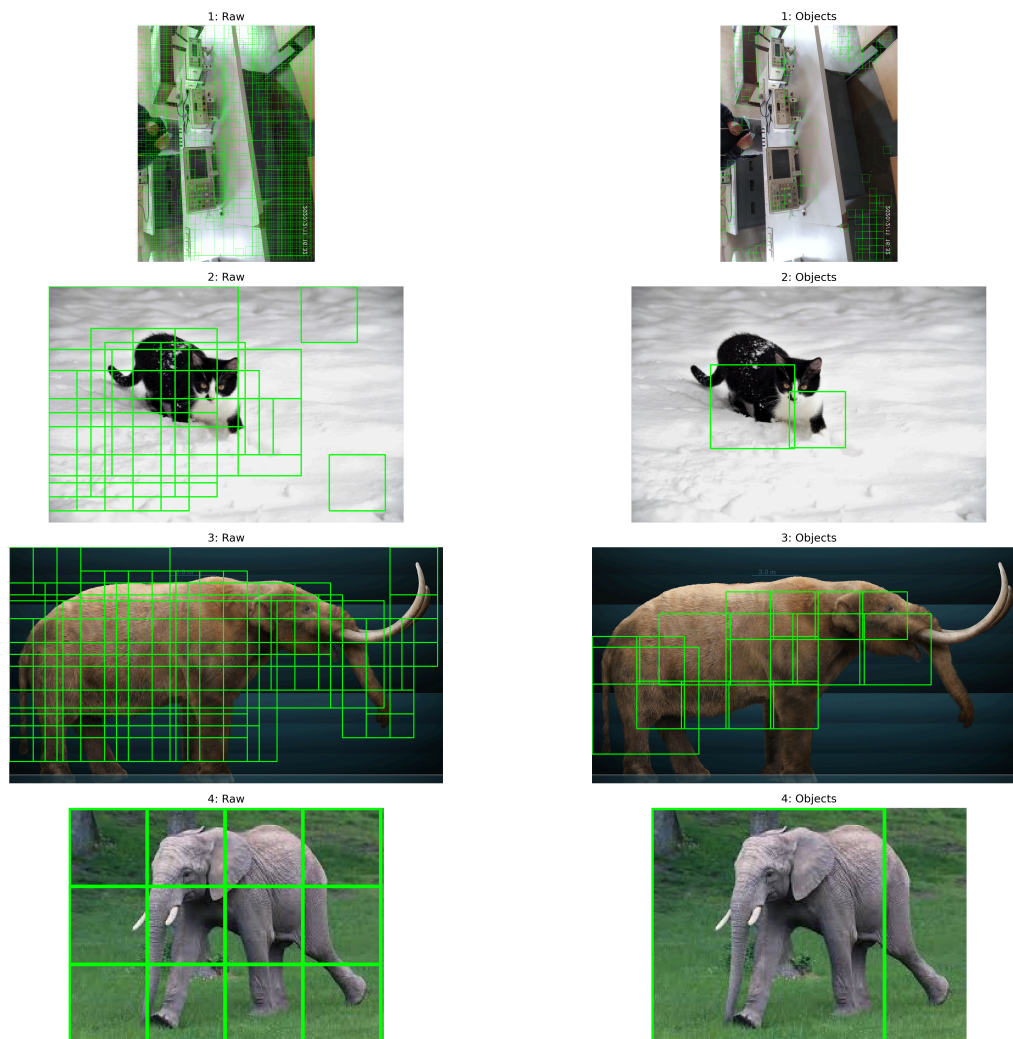
Visualization of Detection Results

Finally, the detected bounding boxes are drawn on the images using rectangles. The project compares two approaches:

- Raw detections without filtering
- Filtered detections after applying NMS

The results are displayed in a grid of images to visually evaluate the effectiveness of the detection pipeline.

Visualization is an important step because it allows manual verification of detection quality.



Conclusion

This project demonstrates a complete classical computer vision pipeline for object detection using **HOG features and SVM classification**. The workflow includes dataset preprocessing, feature extraction, dataset splitting, model evaluation, and object detection using sliding windows and image pyramids.

Although modern deep learning approaches such as CNN-based detectors are more powerful, this traditional approach provides a strong foundation for understanding how object detection systems work internally.

The implemented pipeline highlights the importance of proper data preprocessing, feature representation, and detection filtering techniques in building an effective object detection system.